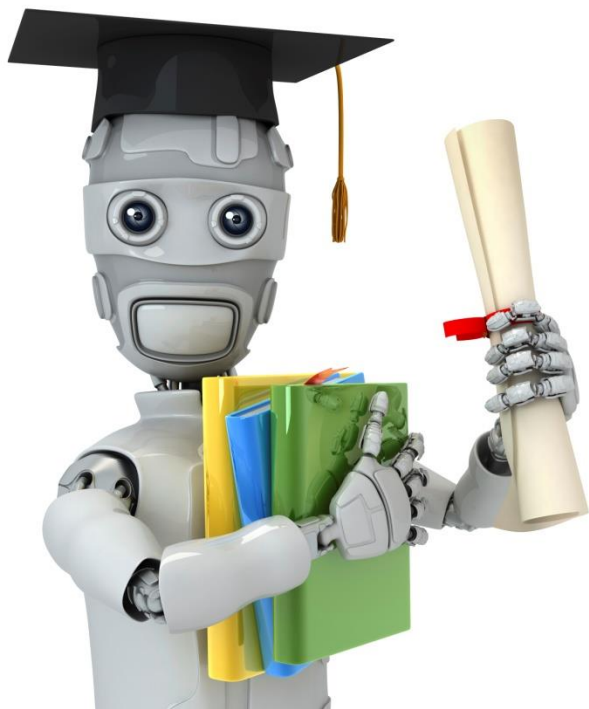


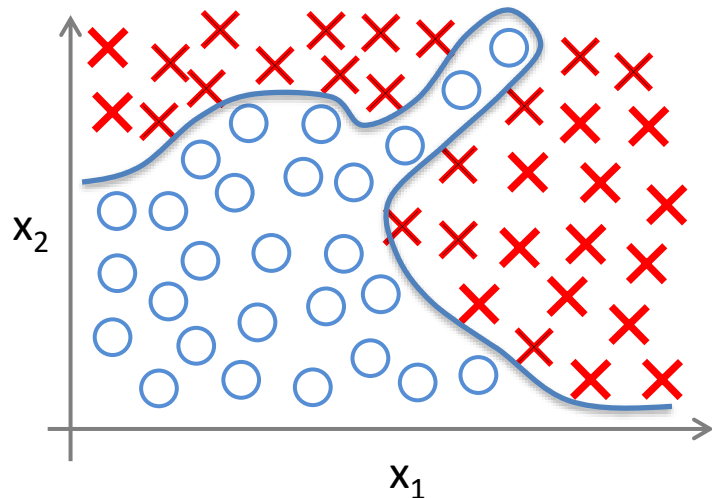


Machine Learning

Neural Networks: Representation

Non-linear hypotheses

Non-linear Classification



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

x_1 = size

x_2 = # bedrooms

x_3 = # floors

x_4 = age

...

x_{100}

$$x_1 x_2 + x_1 x_3 + x_1 x_4 + \dots + x_2 x_3 + x_2 x_4 + \dots + x_{99} x_{100}$$

→ *nearly 5,000 features*

$$x_1 x_2 x_3 + x_1 x_2 x_4 + x_1 x_2 x_5 + \dots + x_2 x_3 x_4 + x_2 x_3 x_5 + \dots + x_{98} x_{99} x_{100}$$

→ *more than 170,000 features!!*

What is this?

You see this:



But the camera sees this:

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

Computer Vision: Car detection



Cars

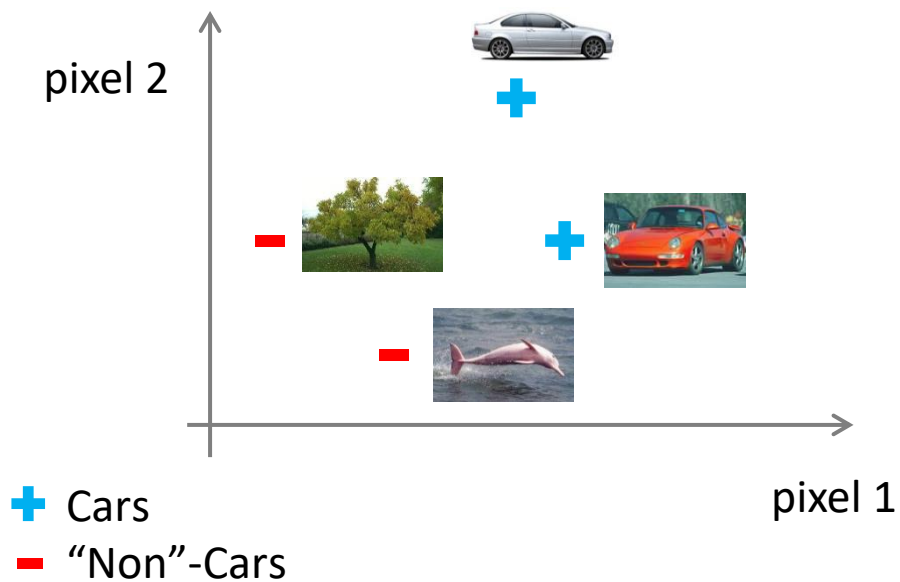
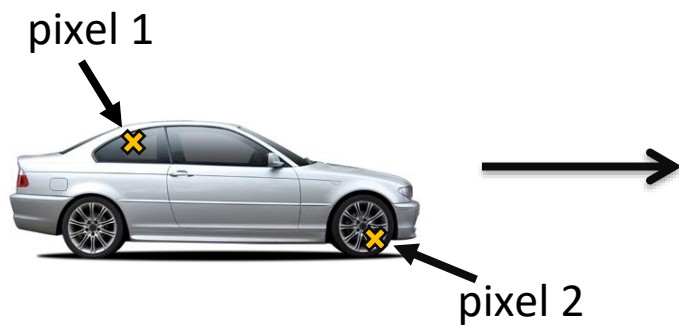


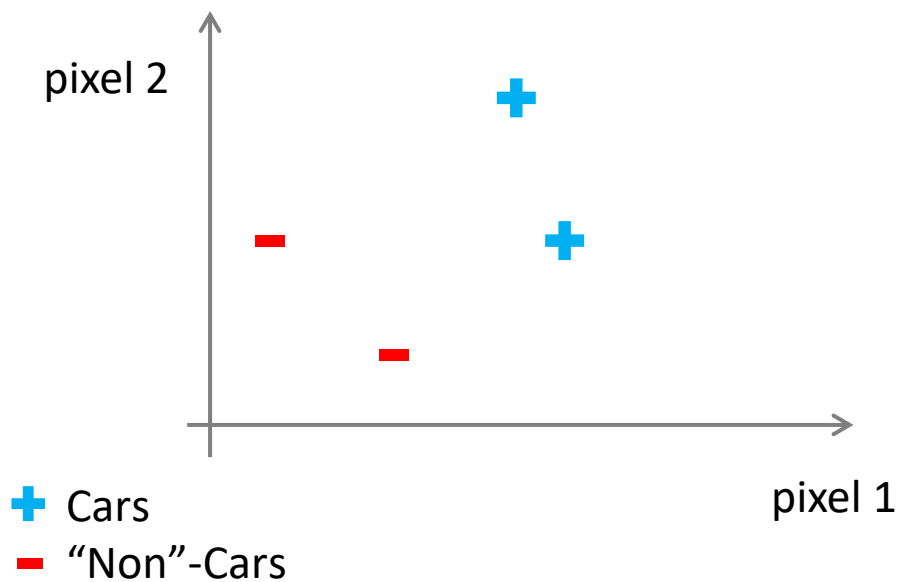
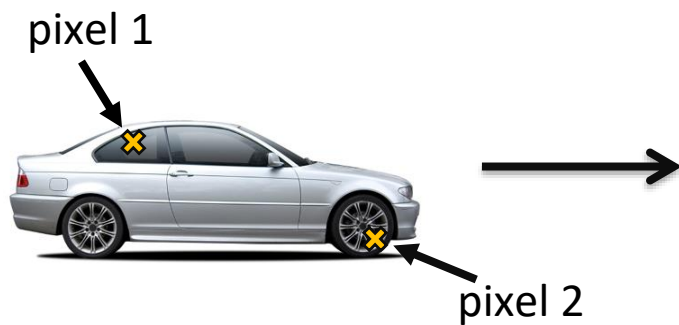
Not a car

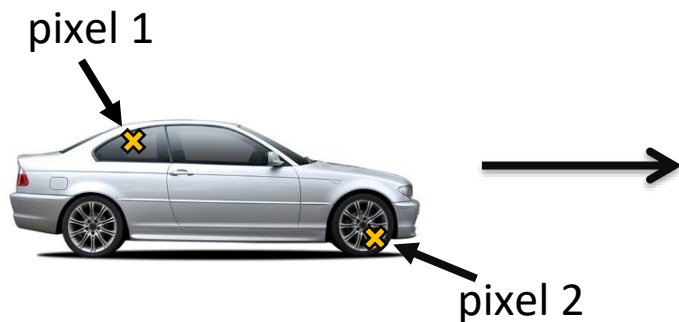
Testing:



What is this?

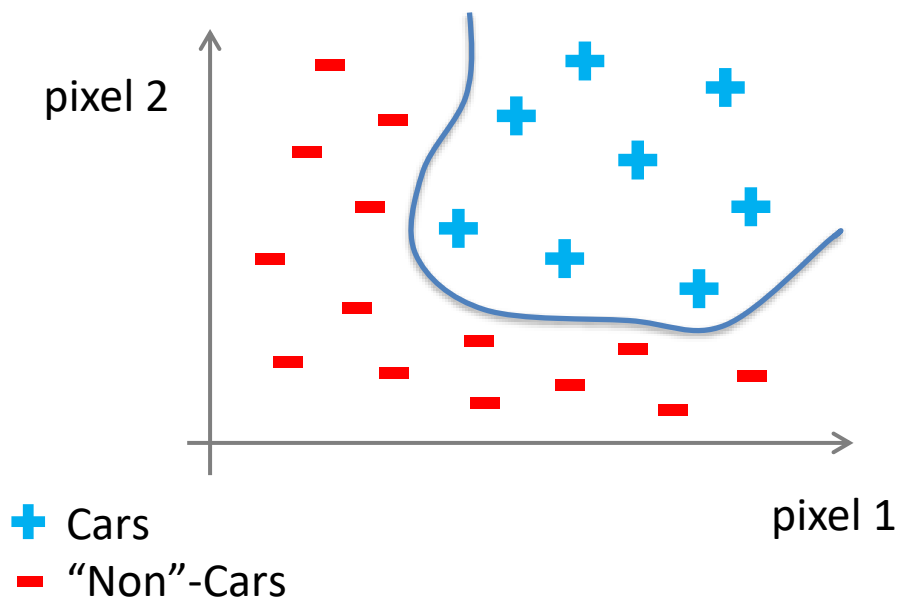






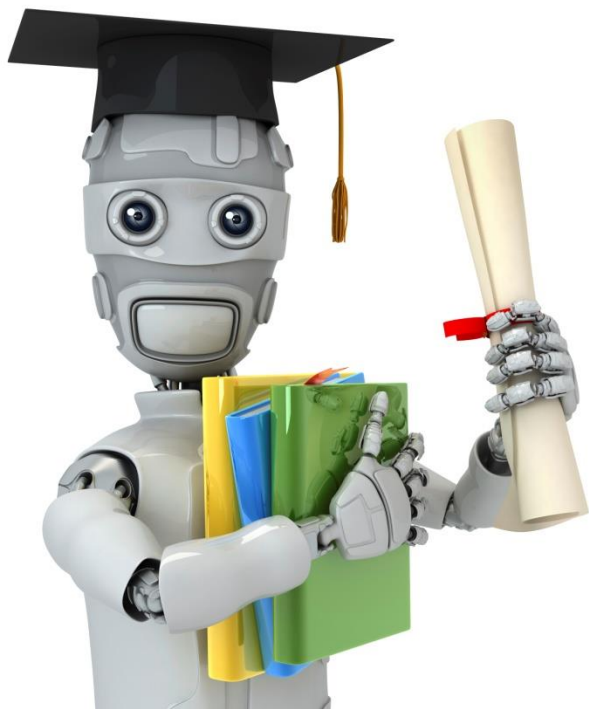
Learning
Algorithm

50 x 50 pixel images $\rightarrow$ 2500 pixels
 $n = 2500$ (7500 if RGB)



$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

Quadratic features ($x_i \times x_j$): ≈ 3 million features



Machine Learning

Neural Networks: Representation

Neurons and the brain

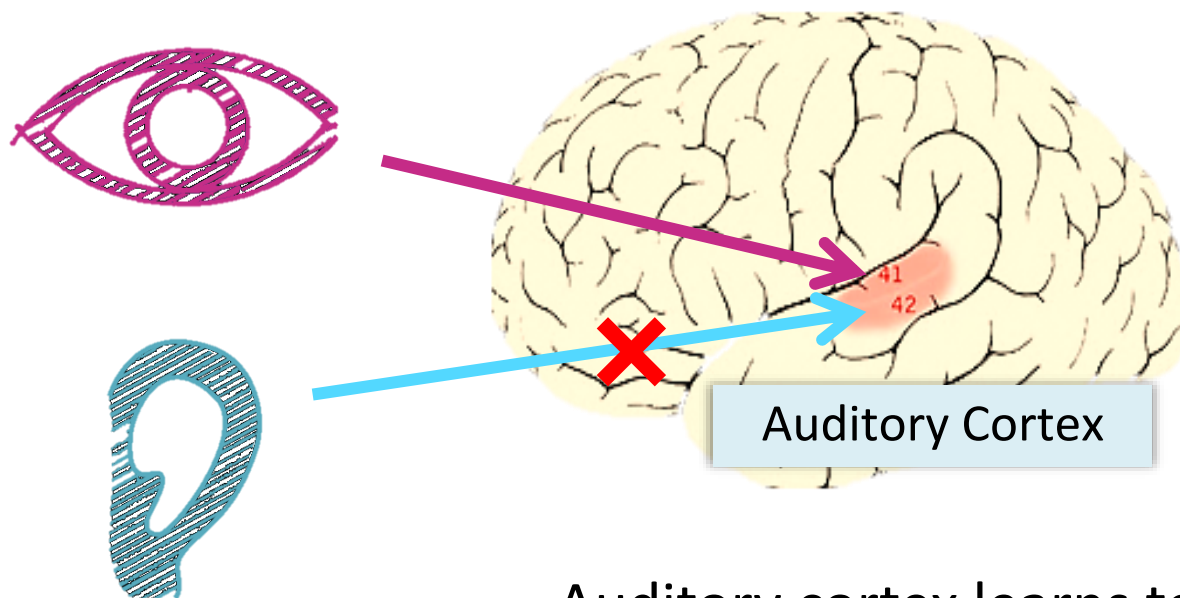
Neural Networks

Origins: Algorithms that try to mimic the brain.

Was very widely used in 80s and early 90s; popularity diminished in late 90s.

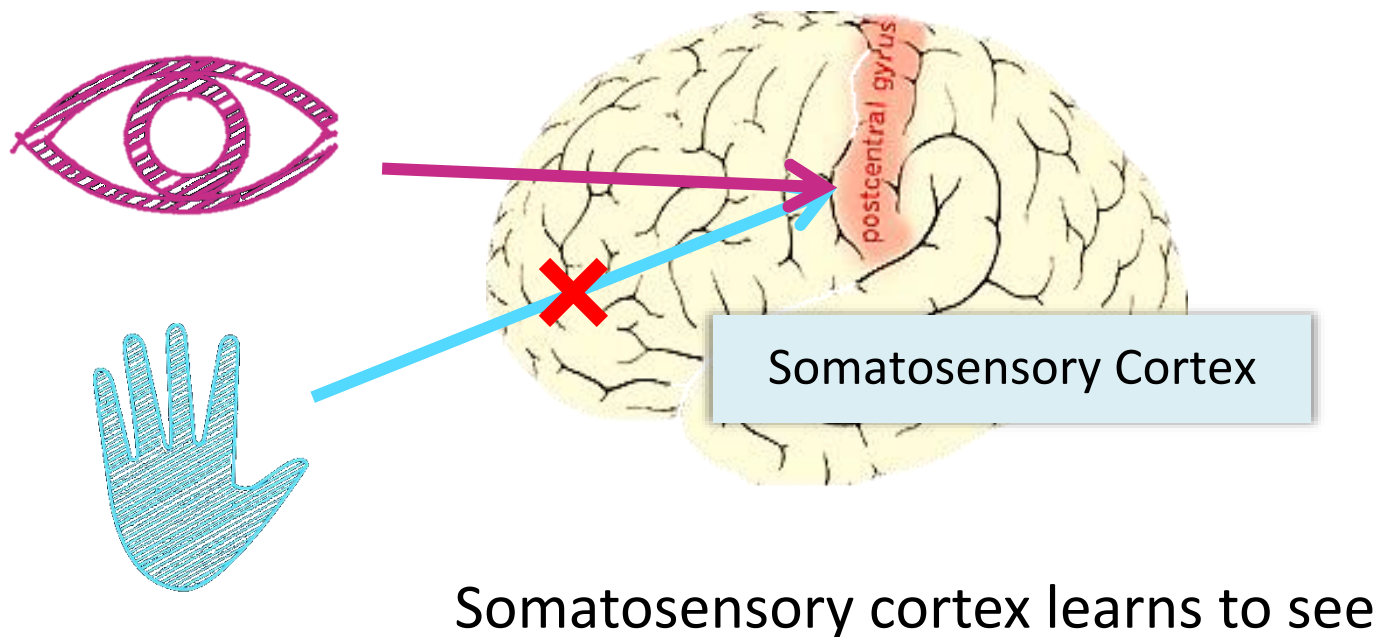
Recent resurgence: State-of-the-art technique for many applications

The “one learning algorithm” hypothesis



Auditory cortex learns to see

The “one learning algorithm” hypothesis



Somatosensory cortex learns to see

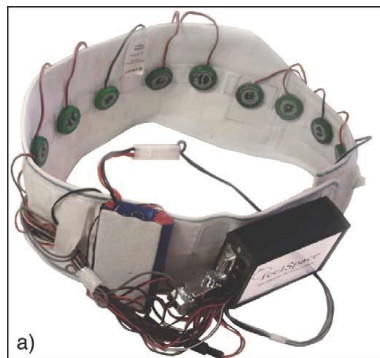
Sensor representations in the brain



Seeing with your tongue



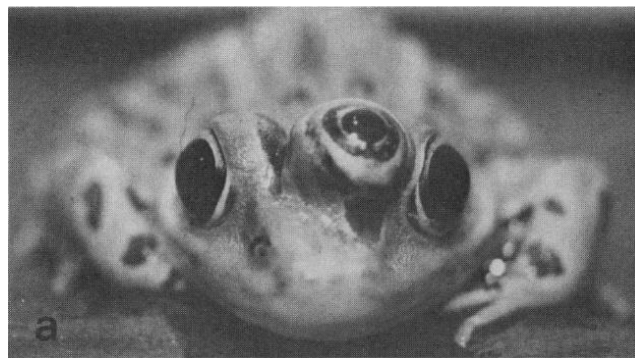
Human echolocation (sonar)



a)

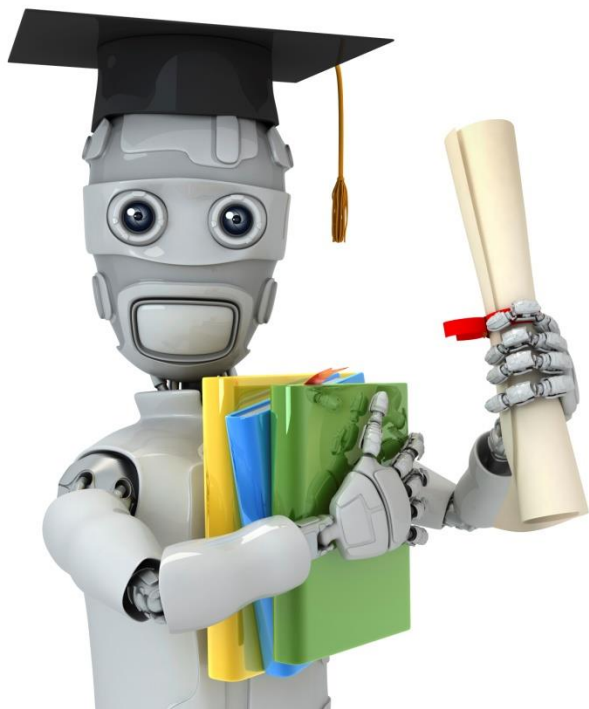


Haptic belt: Direction sense



a

Implanting a 3rd eye

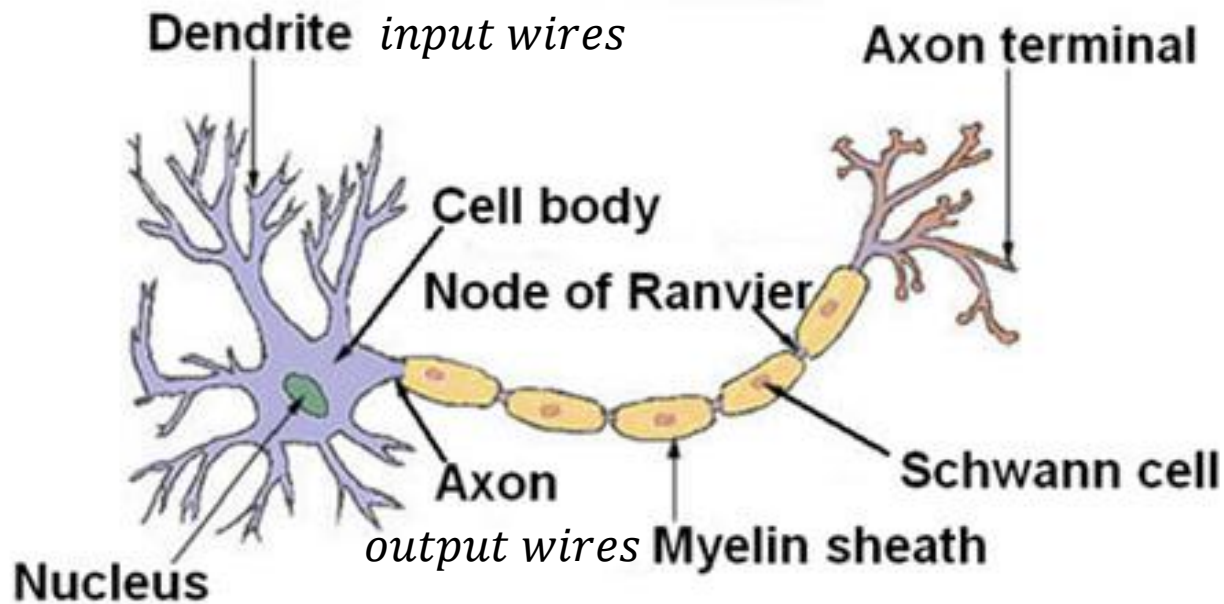


Machine Learning

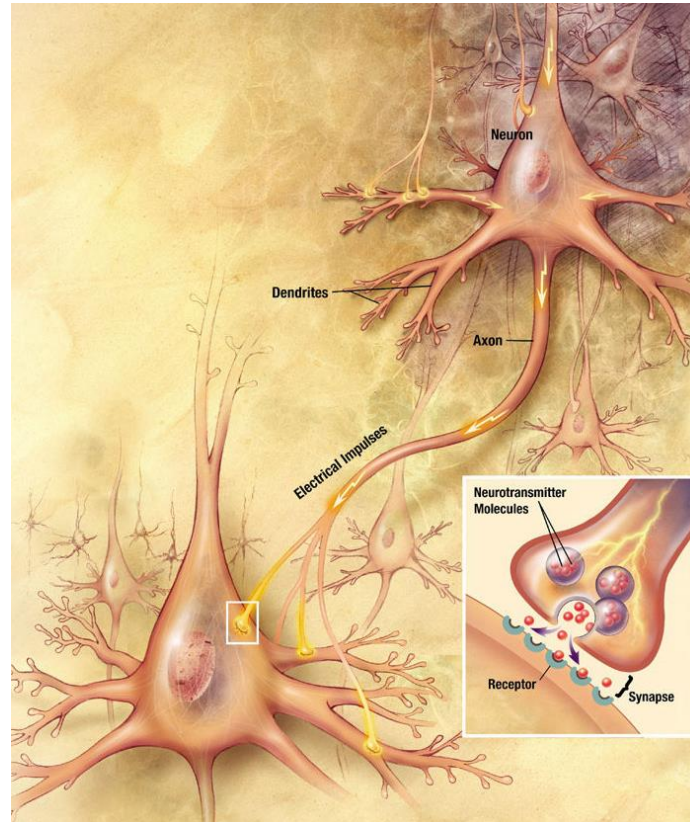
Neural Networks: Representation

Model representation I

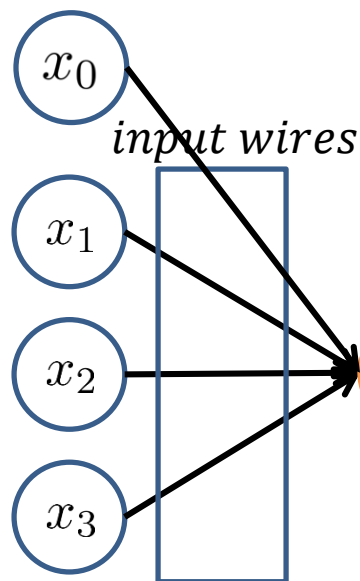
Neuron in the brain



Neurons in the brain



Neuron model: Logistic unit



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

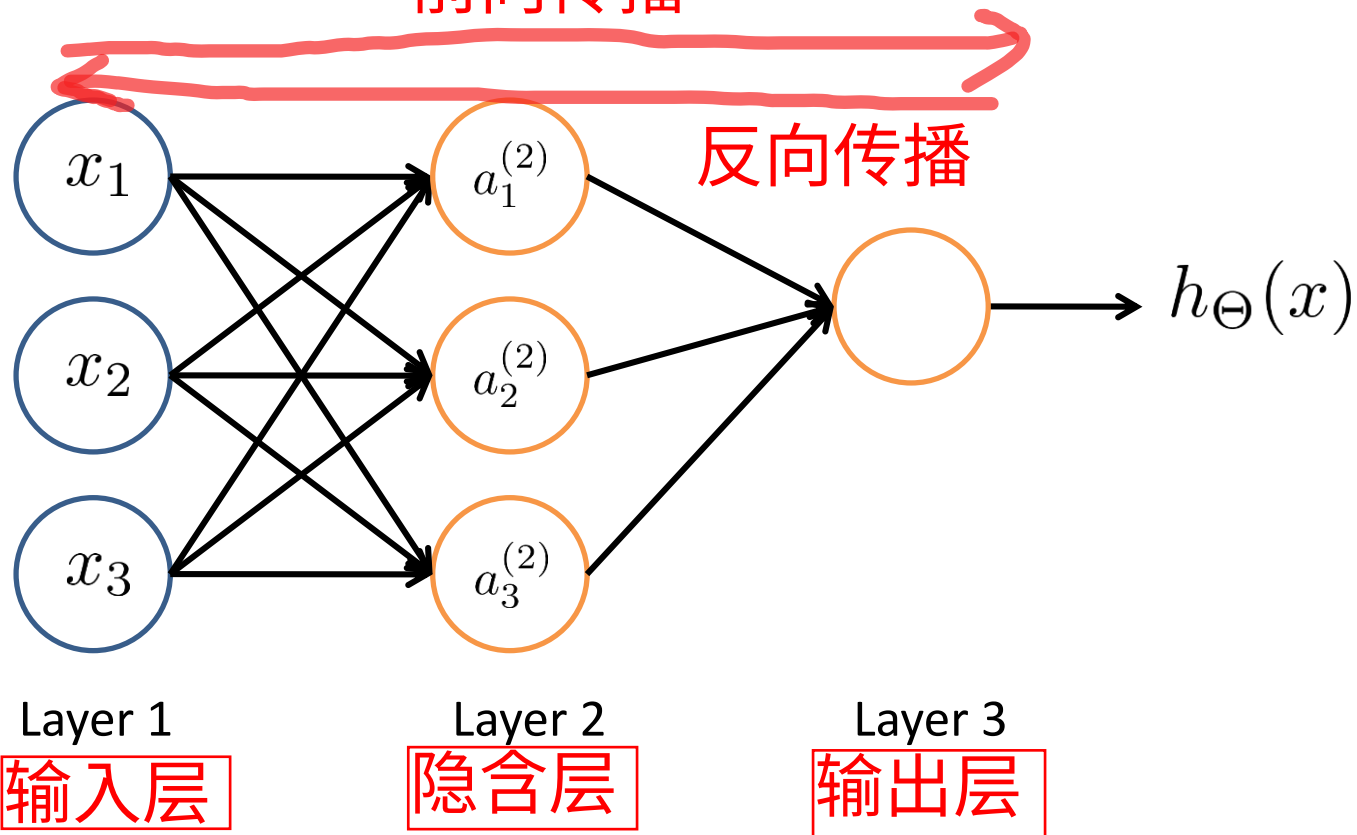
$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Sigmoid (logistic) activation function.

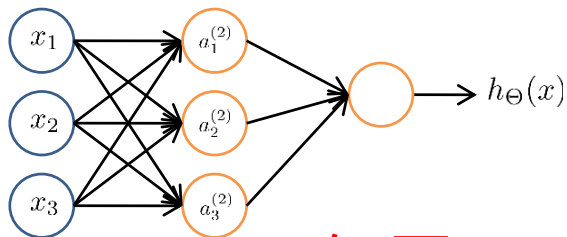
激活函数

Neural Network

前向传播



Neural Network



$a_i^{(j)}$ = “activation” of unit i in layer j

$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer $j + 1$

偏置bias

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

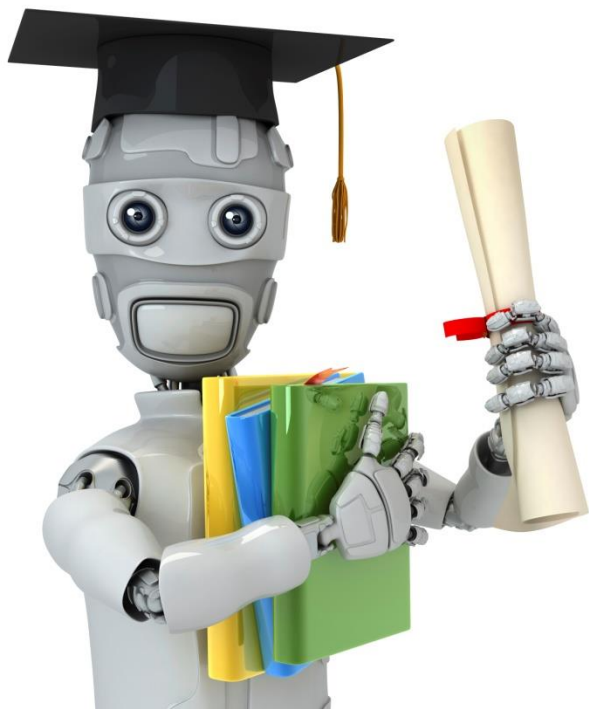
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$\Theta^{(1)}$
 $\Theta^{(2)}$
 $\Theta^{(3)}$ } Θ 矩阵
(parameter weight)

If network has s_j units in layer j , s_{j+1} units in layer $j + 1$, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$. 加上第0个

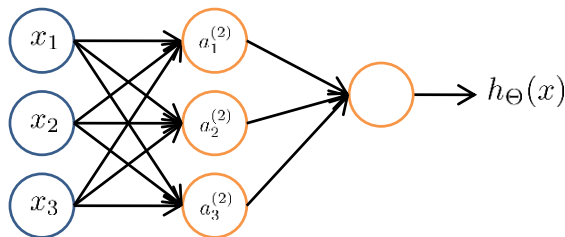


Machine Learning

Neural Networks: Representation

Model representation II

Forward propagation: Vectorized implementation



$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$z^{(2)} = \Theta^{(1)} x$$

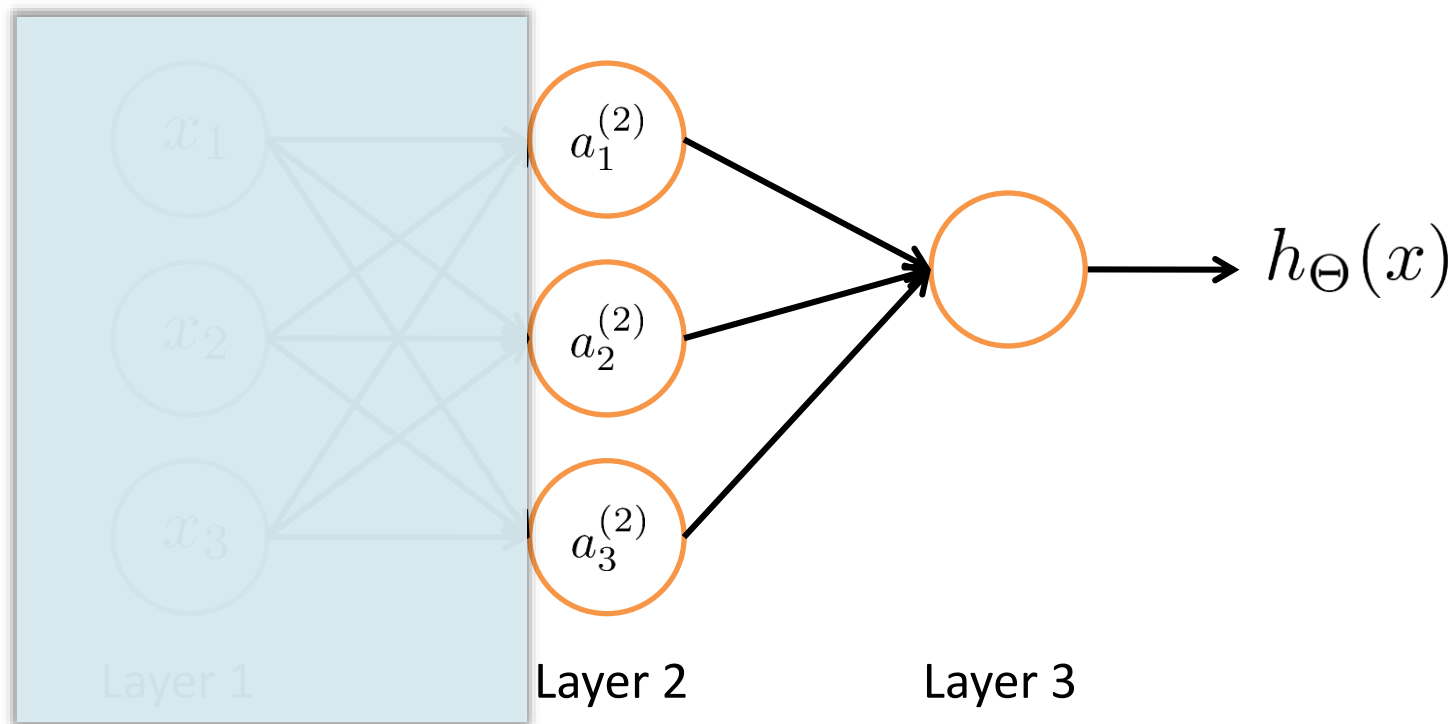
$$a^{(2)} = g(z^{(2)})$$

$$\text{Add } a_0^{(2)} = 1.$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

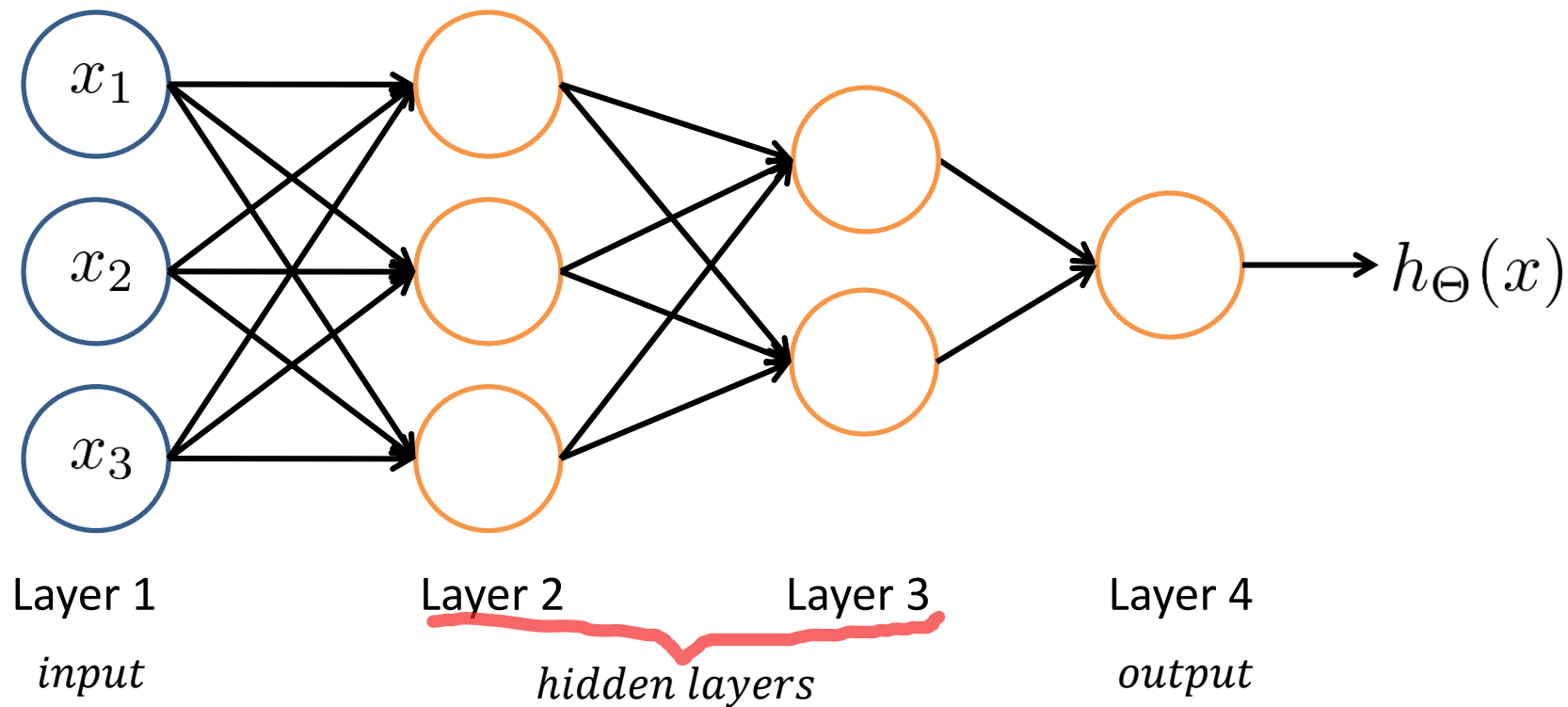
$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)})$$

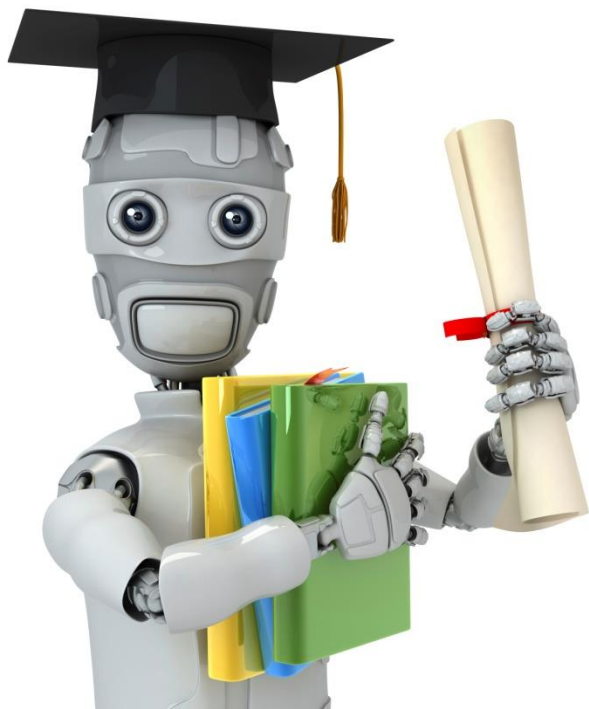
Neural Network learning its own features



$$h_{\theta}(x) = g \left(\theta_0^{(2)} a_0^{(2)} + \theta_1^{(2)} a_1^{(2)} + \theta_2^{(2)} a_2^{(2)} + \theta_3^{(2)} a_3^{(2)} \right)$$

Other network architectures





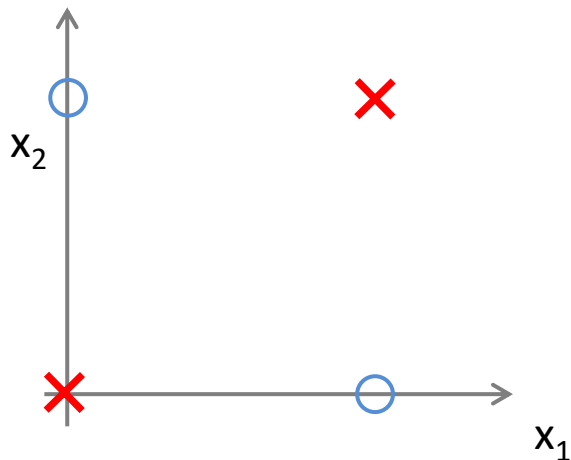
Machine Learning

Neural Networks: Representation

Examples and intuitions I

Non-linear classification example: XOR/XNOR

x_1, x_2 are binary (0 or 1).



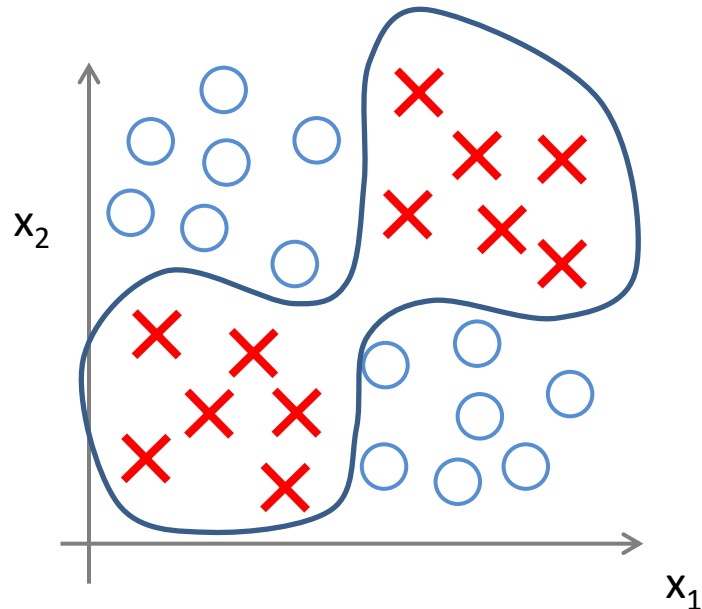
$$y = x_1 \text{ XOR } x_2$$

$$x_1 \text{ XNOR } x_2$$

$$\text{NOT } (x_1 \text{ XOR } x_2)$$

XOR truth table

Input		Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

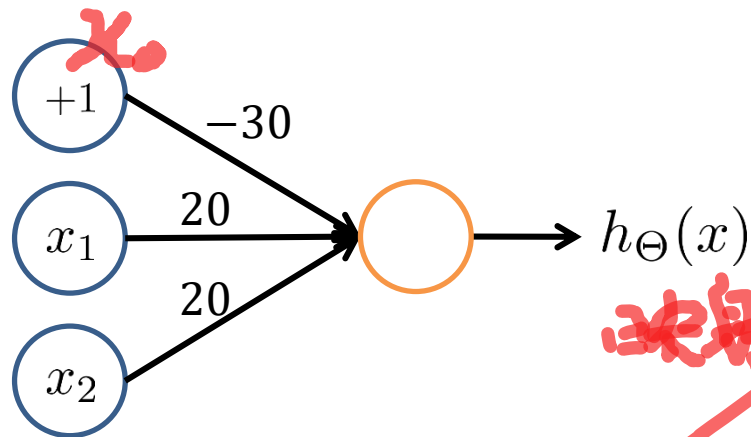


Simple example: AND

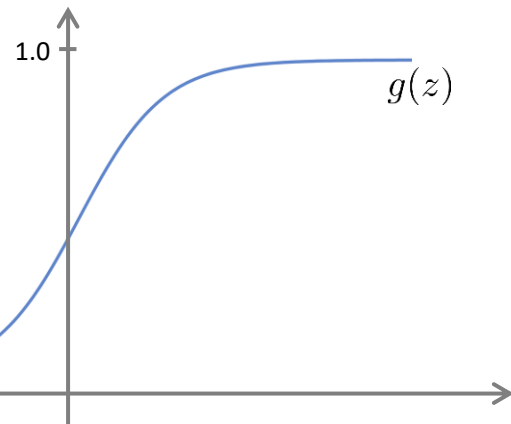
$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$

$$y = g(z)$$



$$g(-30 + 20x_1 + 20x_2)$$

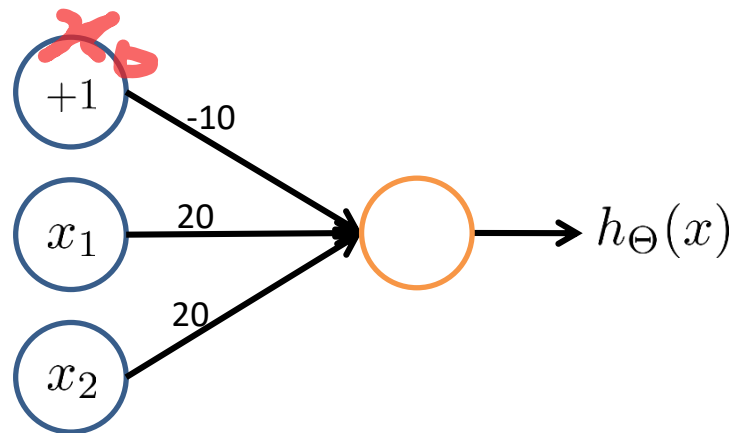


x_1	x_2	$h_{\Theta}(x)$
0	0	≈ 0
0	1	≈ 0
1	0	≈ 0
1	1	≈ 1

$$h_{\Theta}(x) \approx x_1 \text{ AND } x_2$$

Example: OR function

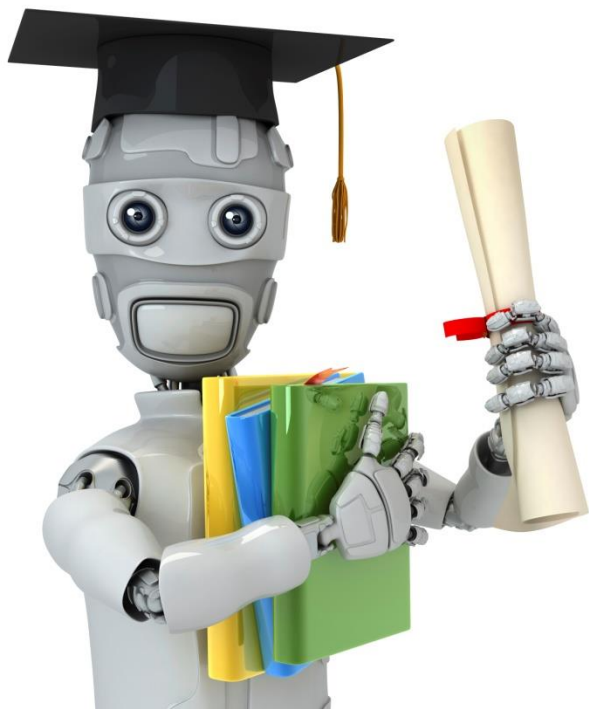
11



$$g(-10 + 20x_1 + 20x_2)$$

x_1	x_2	$h_{\Theta}(x)$
0	0	≈ 0
0	1	≈ 1
1	0	≈ 1
1	1	≈ 1

$$h_{\Theta}(x) \approx x_1 \text{ OR } x_2$$



Machine Learning

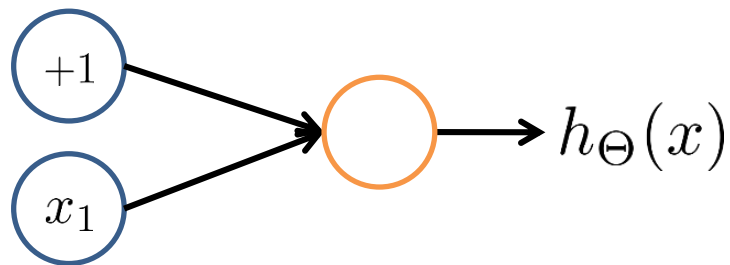
Neural Networks: Representation

Examples and intuitions II

x_1 AND x_2

x_1 OR x_2

Negation:

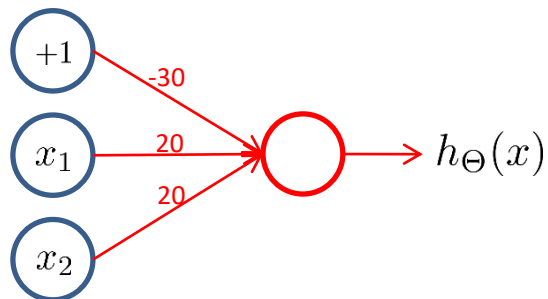


$$h_{\Theta}(x) = g(10 - 20x_1)$$

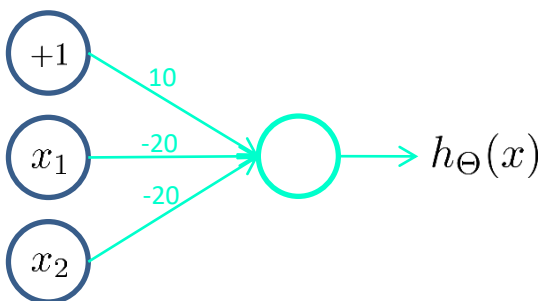
x_1	$h_{\Theta}(x)$
0	≈ 1
1	≈ 0

(NOT x_1) AND (NOT x_2)

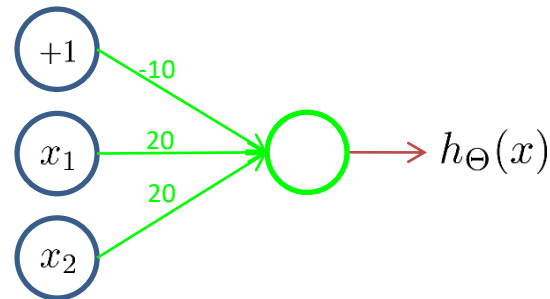
Putting it together: x_1 XNOR x_2



x_1 AND x_2

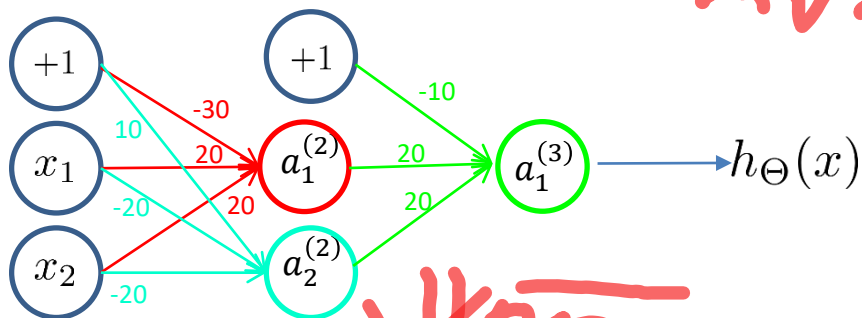


$(\text{NOT } x_1) \text{ AND } (\text{NOT } x_2)$



x_1 OR x_2

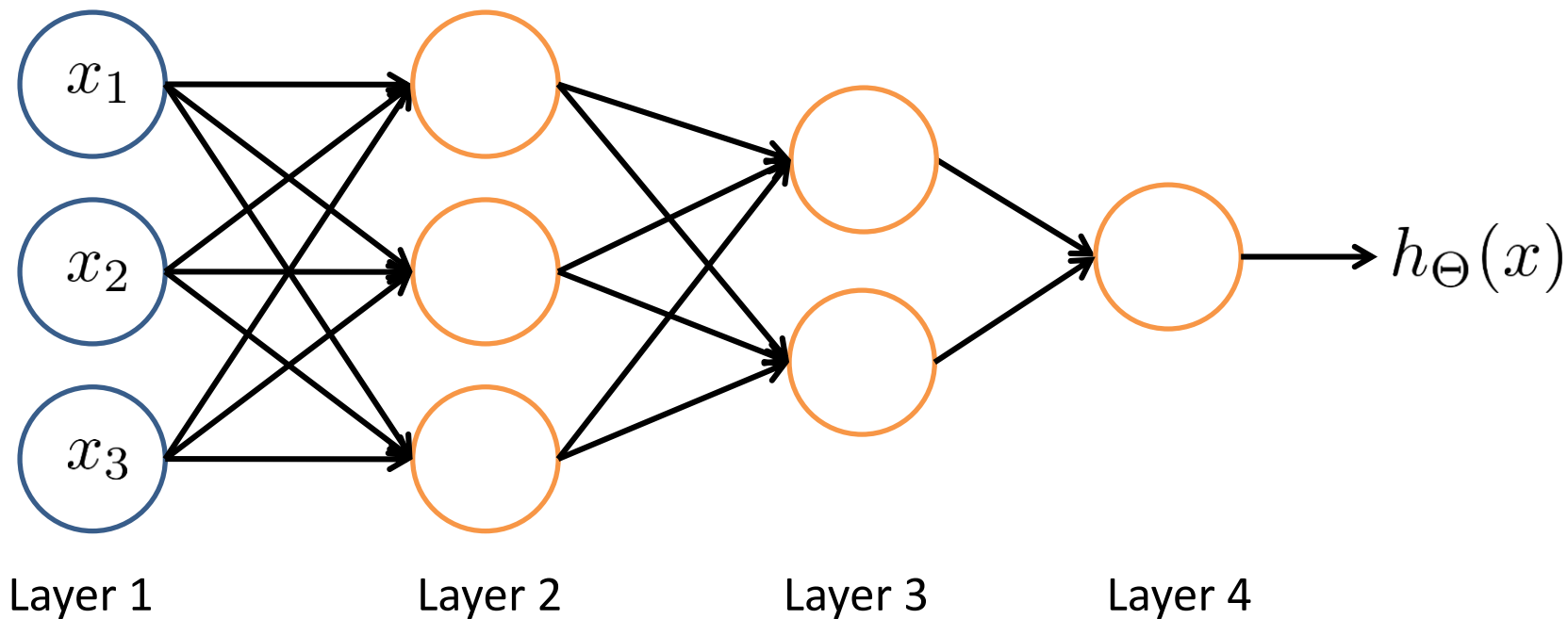
$x_1 \vee x_2$



$x_1 \oplus x_2 = x_1 \oplus x_2$

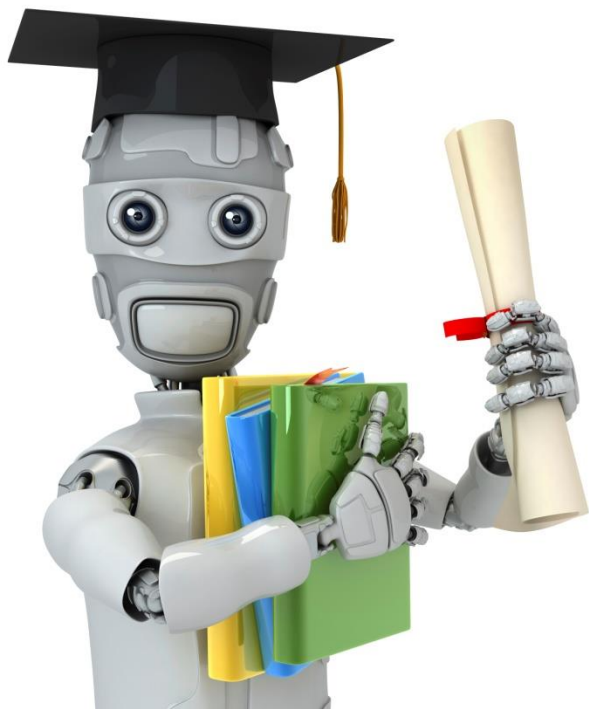
x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Neural Network intuition



Handwritten digit classification





Machine Learning

Neural Networks: Representation

Multi-class classification

Multiple output units: One-vs-all.



Pedestrian



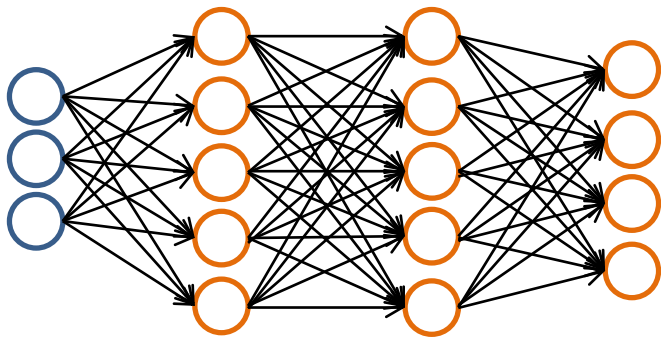
Car



Motorcycle



Truck



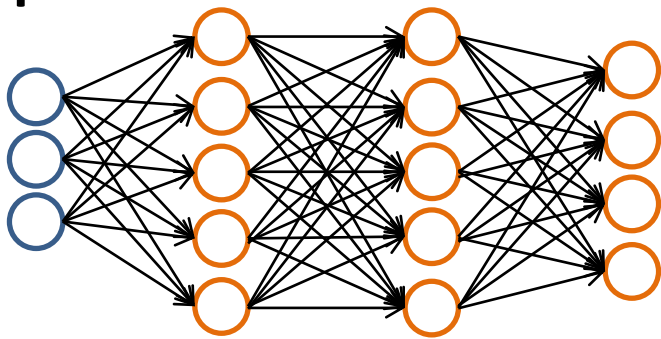
$$h_{\Theta}(x) \in \mathbb{R}^4$$

硬编码

Want $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.

when pedestrian when car when motorcycle

Multiple output units: One-vs-all.



softmax

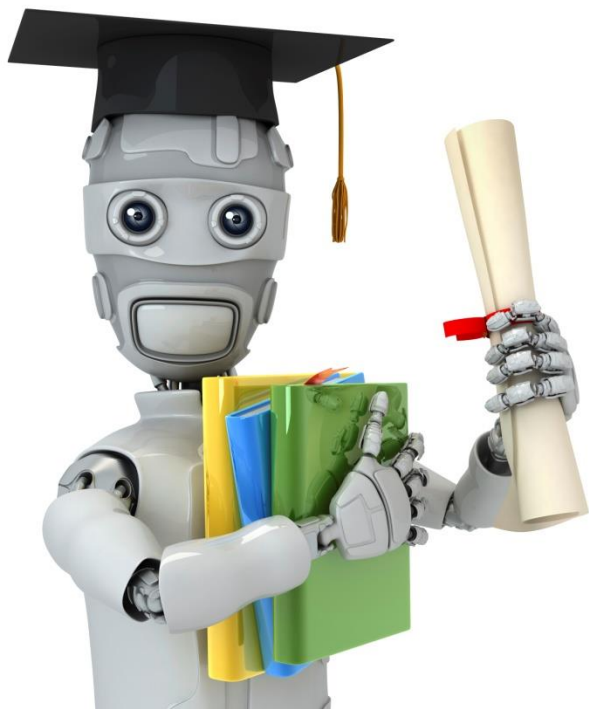
$$h_{\Theta}(x) \in \mathbb{R}^4$$

Want $h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.
when pedestrian when car when motorcycle

Training set: $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

$y^{(i)}$ one of $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$
pedestrian car motorcycle truck

每一行的 y_i 代表每一类的概率，需要将所有的 y_i 进行归一化再选取最大的概率作为分类依据

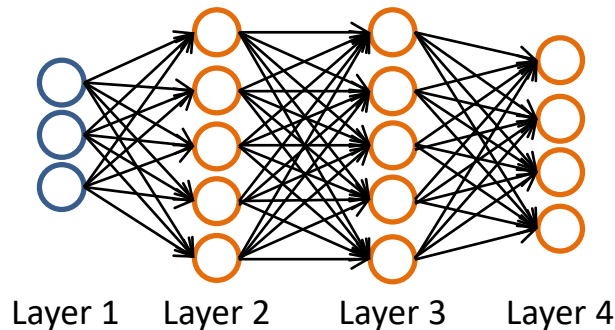


Machine Learning

Neural Networks: Learning

Cost function

Neural Network (Classification)



$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$$

$L =$ total no. of layers in network

$s_l =$ no. of units (not counting bias unit) in layer l

Binary classification

$y = 0$ or 1

1 output unit

Multi-class classification (K classes)

$y \in \mathbb{R}^K$ E.g. $\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$
pedestrian car motorcycle truck

K output units

Cost function

Logistic regression:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

正则化项
↑

Neural network:

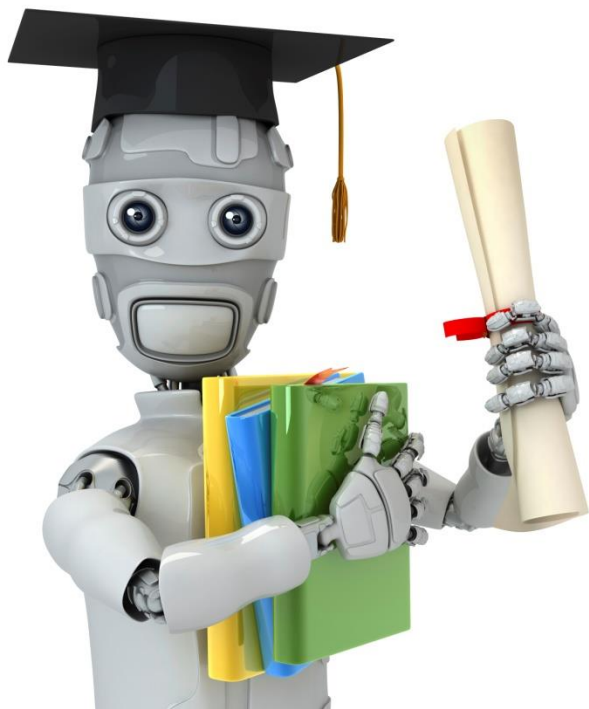
$$h_{\Theta}(x) \in \mathbb{R}^K \quad (h_{\Theta}(x))_i = \text{output}_i$$

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right]$$

$$+ \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

先计算每个样本的每一类的分类损失的和，再统计所有样本的损失和

有多层神经元，每一层神经元之间



Machine Learning

Neural Networks: Learning

Backpropagation algorithm

Gradient computation

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

$$\min_{\Theta} J(\Theta)$$

Need code to compute:

- $J(\Theta)$
 - $\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$
- 上一层的编号
- 对应的神经元的编号
- 当前层的编号

Gradient computation

Given one training example (x, y) :

Forward propagation:

$$a^{(1)} = x$$

$$z^{(2)} = \Theta^{(1)} a^{(1)}$$

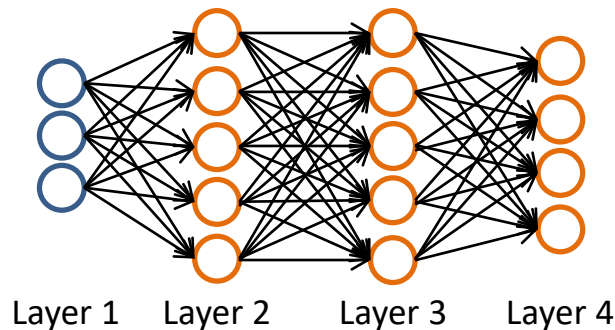
$$a^{(2)} = g(z^{(2)}) \quad (\text{add } a_0^{(2)})$$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)}) \quad (\text{add } a_0^{(3)})$$

$$z^{(4)} = \Theta^{(3)} a^{(3)}$$

$$a^{(4)} = h_{\Theta}(x) = g(z^{(4)})$$



Gradient computation: Backpropagation algorithm

Intuition: $\delta_j^{(l)}$ = “error” of node j in layer l .

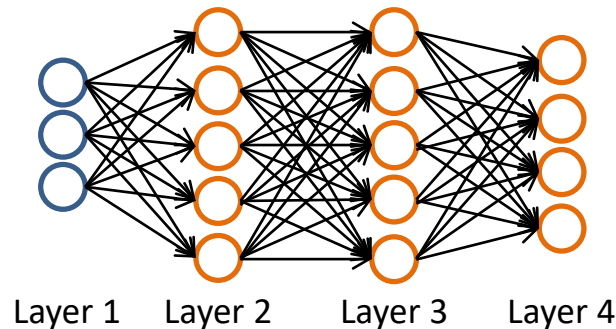
For each output unit (layer L = 4)

$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

$$\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} \overset{\text{点乘}}{\cdot} g'(z^{(3)})$$

$$\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} \cdot g'(z^{(2)})$$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial y}{\partial b} \frac{\partial b}{\partial x}$$



首先由输入正向传播得到输出，所以由反向链式传播可以得到输出经过每一层神经元的偏导

Backpropagation algorithm

Training set $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$

Set $\Delta_{ij}^{(l)} = 0$ (for all l, i, j).

For $i = 1$ to m

Set $a^{(1)} = x^{(i)}$

Perform forward propagation to compute $a^{(l)}$ for $l = 2, 3, \dots, L$

Using $y^{(i)}$, compute $\delta^{(L)} = a^{(L)} - y^{(i)}$

Compute $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$

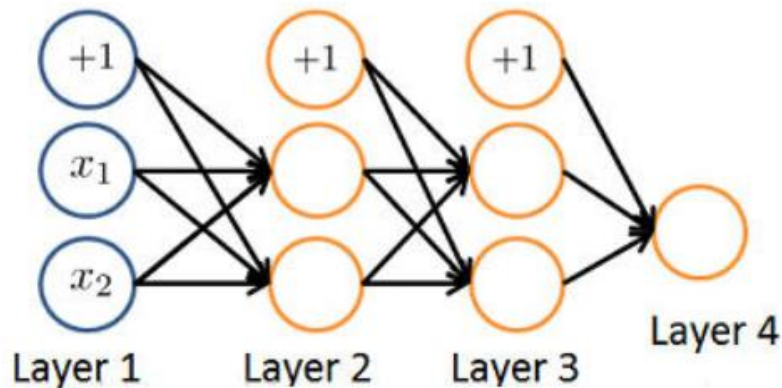
$\Delta_{ij}^{(l)} := \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} + \lambda \Theta_{ij}^{(l)} \text{ if } j \neq 0$$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} \text{ if } j = 0$$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)}$$

考虑下面给出的神经网络。下列哪个方程正确地计算了 $a_1^{(3)}$ 的激活？注： $g(z)$ 是sigmoid激活函数



A. $a_1^{(3)} = g(\Theta_{1,0}^{(2)} a_0^{(2)} + \Theta_{1,1}^{(2)} a_1^{(2)} + \Theta_{1,2}^{(2)} a_2^{(2)})$

B. $a_1^{(3)} = g(\Theta_{1,0}^{(1)} a_0^{(1)} + \Theta_{1,1}^{(1)} a_1^{(1)} + \Theta_{1,2}^{(1)} a_2^{(1)})$

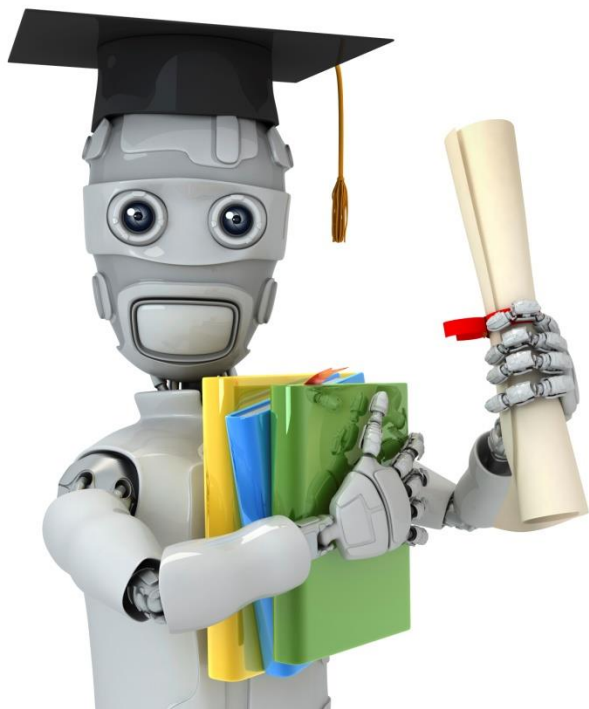
C. $a_1^{(3)} = g(\Theta_{1,0}^{(1)} a_0^{(2)} + \Theta_{1,1}^{(1)} a_1^{(2)} + \Theta_{1,2}^{(1)} a_2^{(2)})$

D. 此网络中不存在激活 $a_1^{(3)}$

您正在训练一个三层神经网络，希望使用反向传播来计算代价函数的梯度。在反向传播算法中，其中一个步骤是更新

$\Delta_{ij}^{(2)} := \Delta_{ij}^{(2)} + \delta_i^{(3)} * (a^{(2)})_j$ 对于每个 i, j ，下面哪一个是这个步骤的正确矢量化？

- A. $\Delta^{(2)} := \Delta^{(2)} + (a^{(2)})^T * \delta^{(3)}$ B. $\Delta^{(2)} := \Delta^{(2)} + (a^{(3)})^T * \delta^{(2)}$
C. $\Delta^{(2)} := \Delta^{(2)} + \delta^{(3)} * (a^{(2)})^T$ D. $\Delta^{(2)} := \Delta^{(2)} + \delta^{(3)} * (a^{(3)})^T$

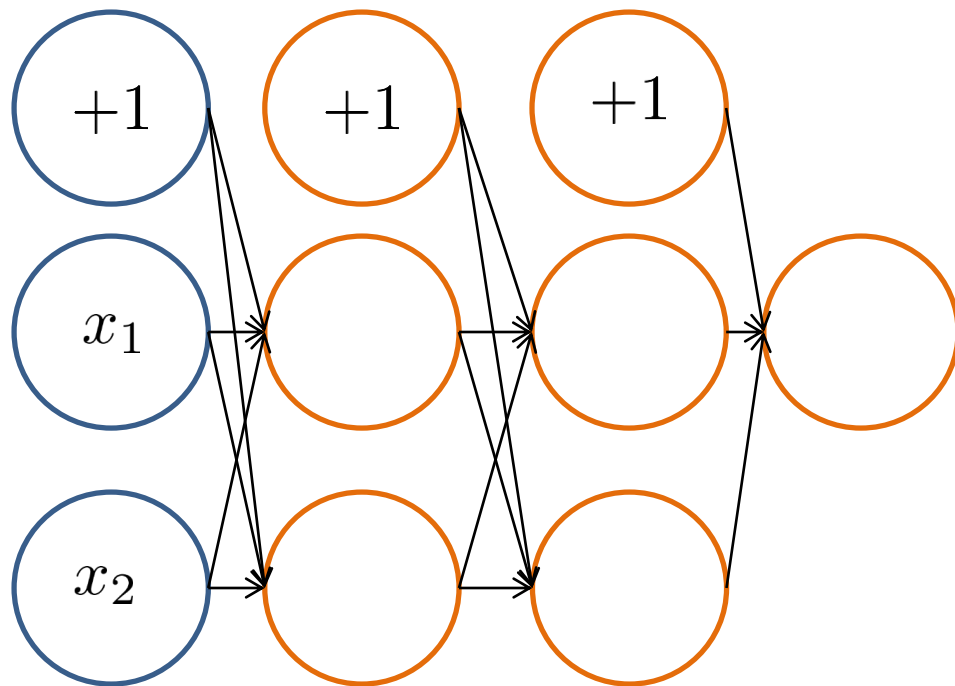


Machine Learning

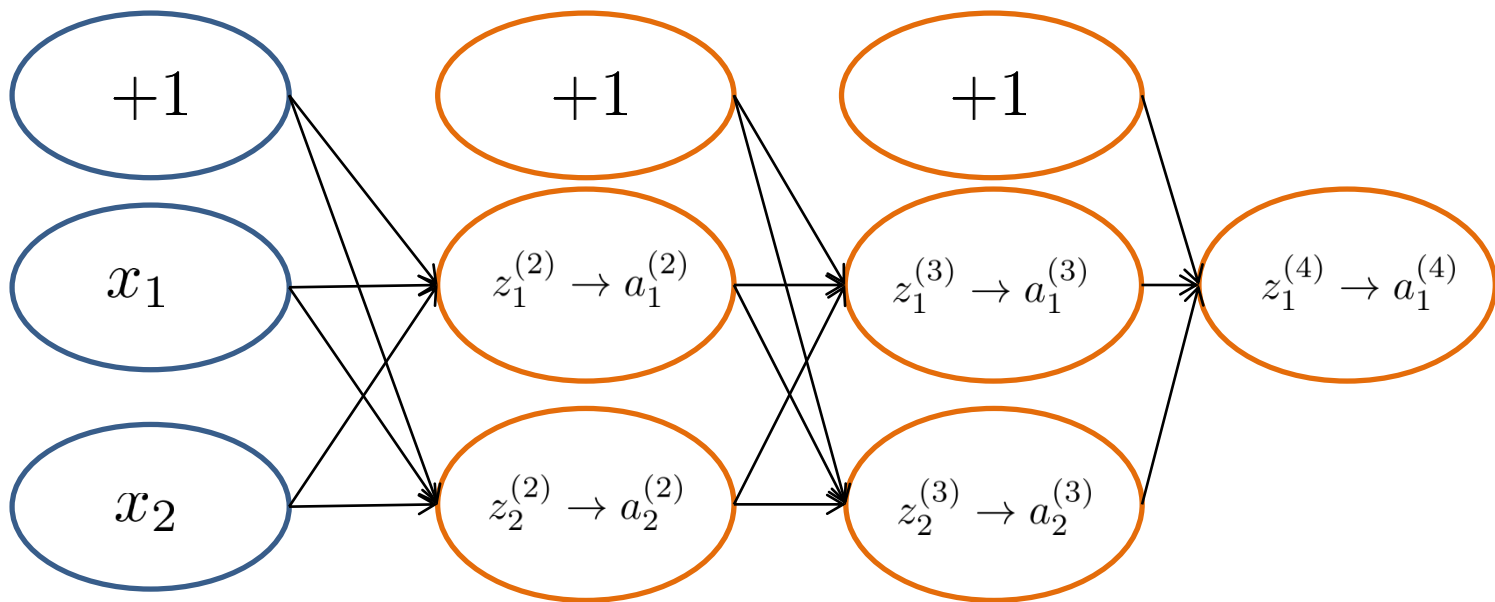
Neural Networks: Learning

Backpropagation intuition

Forward Propagation



Forward Propagation



What is backpropagation doing?

$$J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log(h_{\Theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

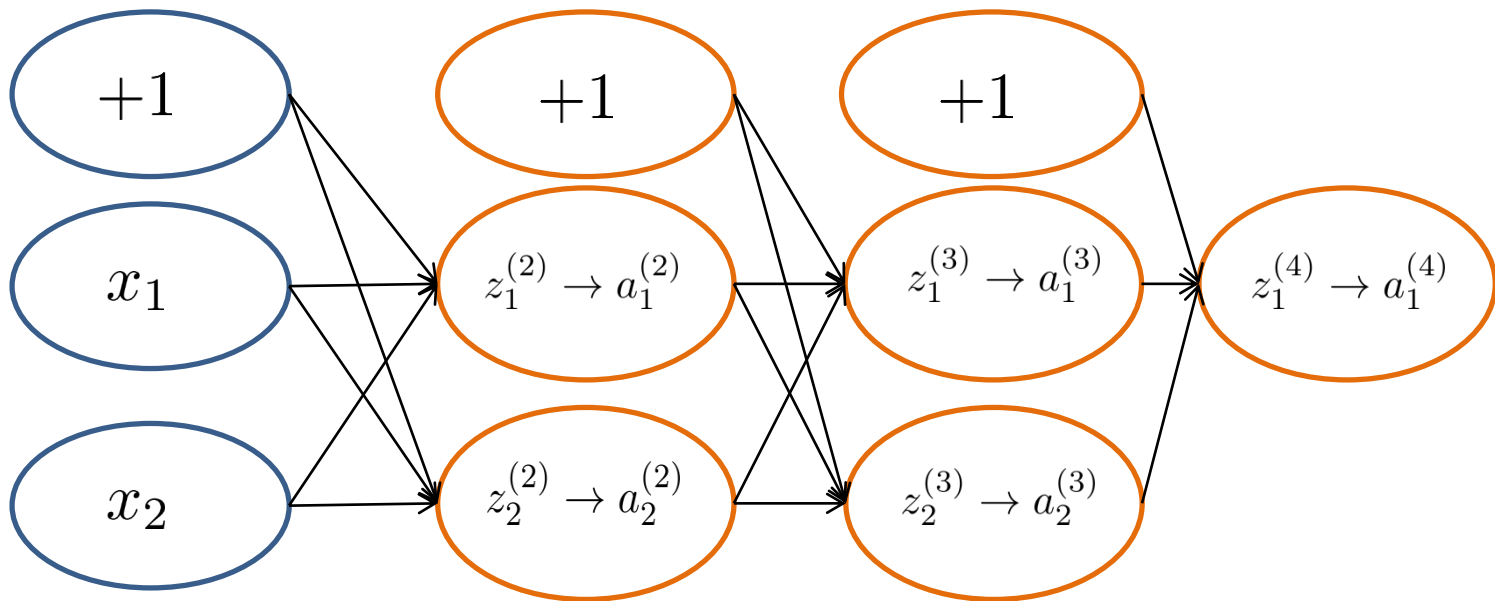
Focusing on a single example $x^{(i)}$, $y^{(i)}$, the case of 1 output unit, and ignoring regularization ($\lambda = 0$),

$$\text{cost}(i) = y^{(i)} \log h_{\Theta}(x^{(i)}) + (1 - y^{(i)}) \log h_{\Theta}(x^{(i)})$$

(Think of $\text{cost}(i) \approx (h_{\Theta}(x^{(i)}) - y^{(i)})^2$)

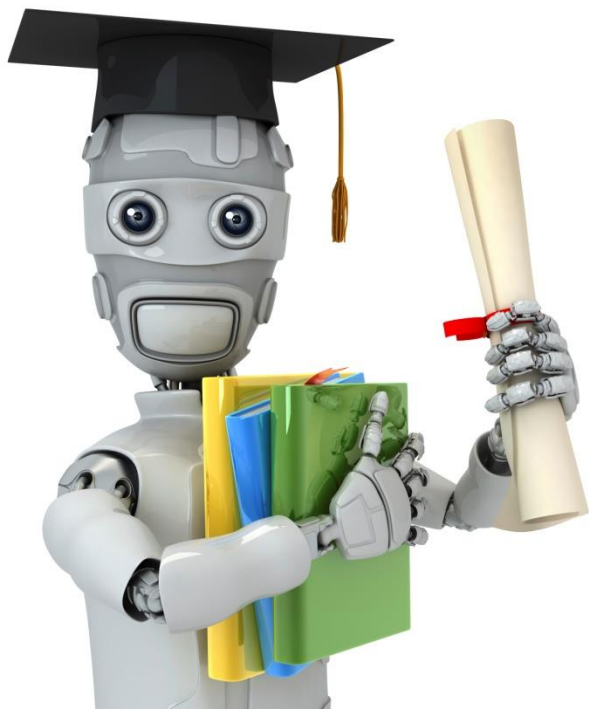
I.e. how well is the network doing on example i ?

Forward Propagation



$\delta_j^{(l)}$ = “error” of cost for $a_j^{(l)}$ (unit j in layer l).

Formally, $\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(i)$ (for $j \geq 0$), where
 $\text{cost}(i) = y^{(i)} \log h_{\Theta}(x^{(i)}) + (1 - y^{(i)}) \log h_{\Theta}(x^{(i)})$



Machine Learning

Neural Networks: Learning

Implementation note:
Unrolling parameters

Advanced optimization

```
function [jVal, gradient] = costFunction(theta)
```

```
...
```

```
optTheta = fminunc(@costFunction, initialTheta, options)
```

Neural Network (L=4):

$\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$ - matrices (**Theta1, Theta2, Theta3**)

$D^{(1)}, D^{(2)}, D^{(3)}$ - matrices (**D1, D2, D3**)

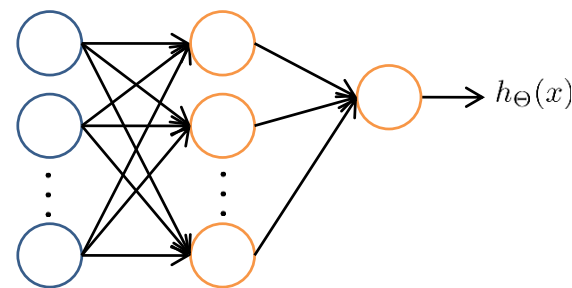
“Unroll” into vectors

Example

$$s_1 = 10, s_2 = 10, s_3 = 1$$

$$\Theta^{(1)} \in \mathbb{R}^{10 \times 11}, \Theta^{(2)} \in \mathbb{R}^{10 \times 11}, \Theta^{(3)} \in \mathbb{R}^{1 \times 11}$$

$$D^{(1)} \in \mathbb{R}^{10 \times 11}, D^{(2)} \in \mathbb{R}^{10 \times 11}, D^{(3)} \in \mathbb{R}^{1 \times 11}$$



```
thetaVec = [ Theta1(:) ; Theta2(:) ; Theta3(:) ] ;
```

```
DVec = [ D1(:) ; D2(:) ; D3(:) ] ;
```

```
Theta1 = reshape(thetaVec(1:110), 10, 11) ;
```

```
Theta2 = reshape(thetaVec(111:220), 10, 11) ;
```

```
Theta3 = reshape(thetaVec(221:231), 1, 11) ;
```

Learning Algorithm

Have initial parameters $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$.

Unroll to get `initialTheta` to pass to

`fminunc(@costFunction, initialTheta, options)`

`function [jval, gradientVec] = costFunction(thetaVec)`

From `thetaVec`, get $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$.

Use forward prop/back prop to compute $D^{(1)}, D^{(2)}, D^{(3)}$ and $J(\Theta)$.

Unroll $D^{(1)}, D^{(2)}, D^{(3)}$ to get `gradientVec`.

假设Theta1是一个5x3矩阵, Theta2是一个4x6矩阵。令:

`thetaVec=[Theta1(:);Theta2(:)]`。下列哪一项可以正确地还原Theta2?

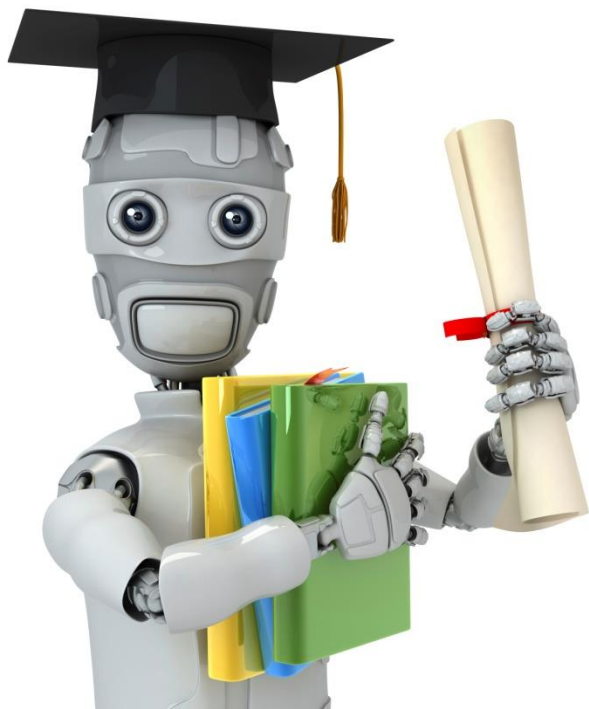
A. `reshape(thetaVec(16:39),4,6)`

B. `reshape(thetaVec(15:38),4,6)`

C. `reshape(thetaVec(16:24),4,6)`

D. `reshape(thetaVec(15:39),4,6)`

E. `reshape(thetaVec(16:39),6,4)`

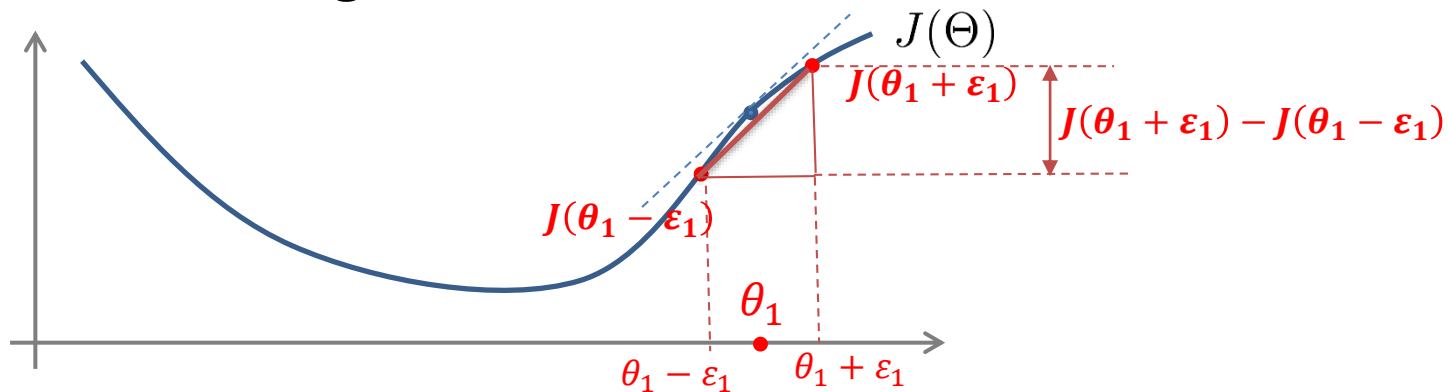


Machine Learning

Neural Networks: Learning

Gradient checking

Numerical estimation of gradients



$$\frac{\partial}{\partial \theta_1} = \frac{J(\theta_1 + \epsilon_1) - J(\theta_1 - \epsilon_1)}{2\epsilon} \quad \epsilon > 0$$

Parameter vector θ

$\theta \in \mathbb{R}^n$ (E.g. θ is “unrolled” version of $\Theta^{(1)}, \Theta^{(2)}, \Theta^{(3)}$)

$$\theta = \theta_1, \theta_2, \theta_3, \dots, \theta_n$$

$$\frac{\partial}{\partial \theta_1} J(\theta) \approx \frac{J(\theta_1 + \epsilon, \theta_2, \theta_3, \dots, \theta_n) - J(\theta_1 - \epsilon, \theta_2, \theta_3, \dots, \theta_n)}{2\epsilon}$$

$$\frac{\partial}{\partial \theta_2} J(\theta) \approx \frac{J(\theta_1, \theta_2 + \epsilon, \theta_3, \dots, \theta_n) - J(\theta_1, \theta_2 - \epsilon, \theta_3, \dots, \theta_n)}{2\epsilon}$$

$\vdots$

$$\frac{\partial}{\partial \theta_n} J(\theta) \approx \frac{J(\theta_1, \theta_2, \theta_3, \dots, \theta_n + \epsilon) - J(\theta_1, \theta_2, \theta_3, \dots, \theta_n - \epsilon)}{2\epsilon}$$

```
for i = 1:n,  
    thetaPlus = theta;  
    thetaPlus(i) = thetaPlus(i) + EPSILON;  
    thetaMinus = theta;  
    thetaMinus(i) = thetaMinus(i) - EPSILON;  
    gradApprox(i) = (J(thetaPlus) - J(thetaMinus))  
                    / (2*EPSILON);  
end;
```

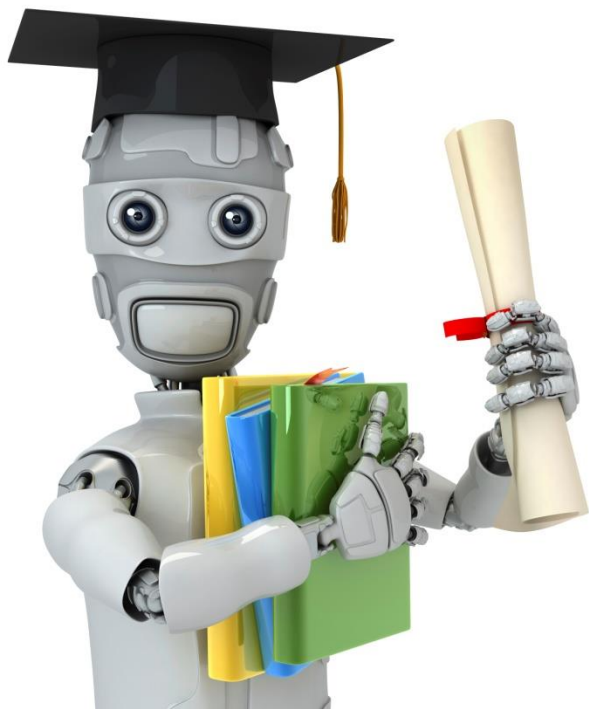
Check that $\text{gradApprox} \approx \text{DVec}$

Implementation Note:

- Implement backprop to compute **DVec** (unrolled $D^{(1)}, D^{(2)}, D^{(3)}$).
- Implement numerical gradient check to compute **gradApprox**.
- Make sure they give similar values.
- Turn off gradient checking. Using backprop code for learning.

Important:

- Be sure to disable your gradient checking code before training your classifier. If you run numerical gradient computation on every iteration of gradient descent (or in the inner loop of **costFunction(...)**) your code will be very slow.



Machine Learning

Neural Networks: Learning

Random initialization

Initial value of Θ

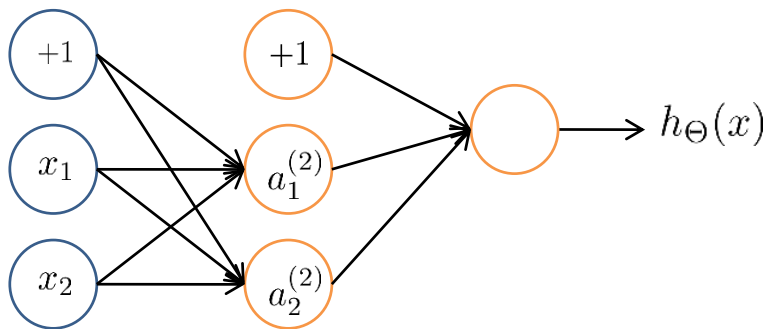
For gradient descent and advanced optimization method, need initial value for Θ .

```
optTheta = fminunc(@costFunction,  
                  initialTheta, options)
```

Consider gradient descent

Set `initialTheta = zeros(n,1)` ?

Zero initialization



$$\Theta_{ij}^{(l)} = 0 \text{ for all } i, j, l.$$

After each update, parameters corresponding to inputs going into each of two hidden units are identical.

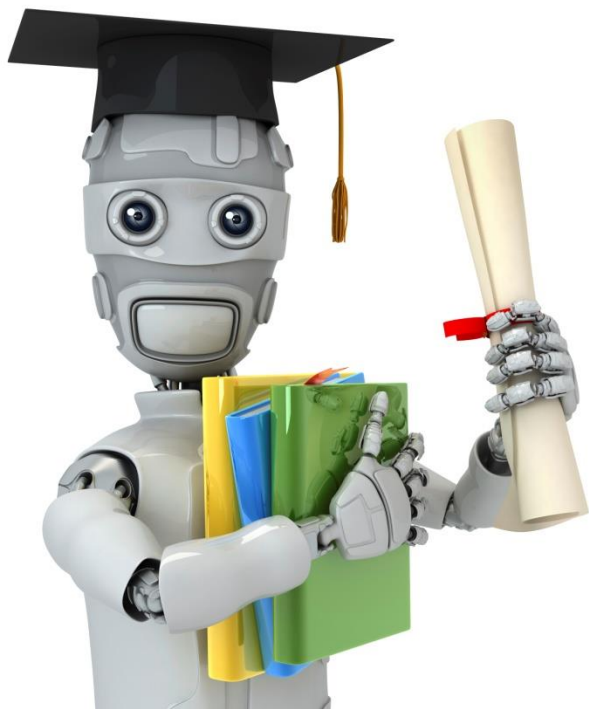
Random initialization: Symmetry breaking

Initialize each $\Theta_{ij}^{(l)}$ to a random value in $[-\epsilon, \epsilon]$
(i.e. $-\epsilon \leq \Theta_{ij}^{(l)} \leq \epsilon$)

E.g.

```
Theta1 = rand(10,11) * (2*INIT_EPSILON)
        - INIT_EPSILON;
```

```
Theta2 = rand(1,11) * (2*INIT_EPSILON)
        - INIT_EPSILON;
```



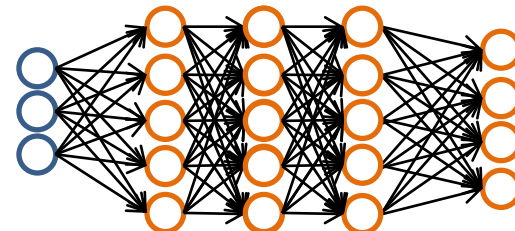
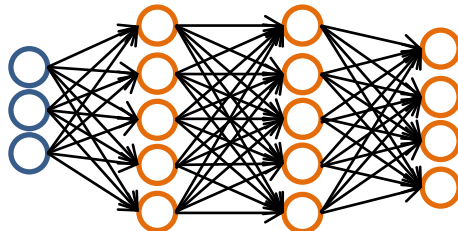
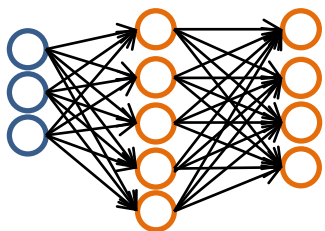
Machine Learning

Neural Networks: Learning

Putting it together

Training a neural network

Pick a network architecture (connectivity pattern between neurons)



No. of input units: Dimension of features $x^{(i)}$

No. output units: Number of classes

Reasonable default: 1 hidden layer, or if >1 hidden layer, have same no. of hidden units in every layer (usually the more the better)

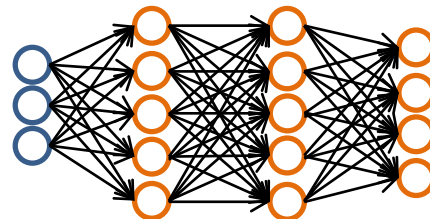
Training a neural network

1. Randomly initialize weights
2. Implement forward propagation to get $h_{\Theta}(x^{(i)})$ for any $x^{(i)}$
3. Implement code to compute cost function $J(\Theta)$
4. Implement backprop to compute partial derivatives $\frac{\partial}{\partial \Theta_{jk}^{(l)}} J(\Theta)$

for $i = 1:m$

 Perform forward propagation and backpropagation using
 example $(x^{(i)}, y^{(i)})$

 (Get activations $a^{(l)}$ and delta terms $\delta^{(l)}$ for $l = 2, \dots, L$).



Training a neural network

5. Use gradient checking to compare $\frac{\partial}{\partial \Theta_{jk}^{(l)}} J(\Theta)$ computed using backpropagation vs. using numerical estimate of gradient of $J(\Theta)$.

Then disable gradient checking code.

6. Use gradient descent or advanced optimization method with backpropagation to try to minimize $J(\Theta)$ as a function of parameters Θ

